

7. Multimodal Visual Localization Voice Interaction

7. Multimodal Visual Localization Voice Interaction

1. Concept Introduction
 - 1.1 What is "Multimodal Visual Localization"?
 - 1.2 Brief Description of Implementation Principles
2. Code Analysis
 - Key Code
 1. Tool Layer Entry (`largemodel/utils/tools_manager.py`)
 2. Model Interface Layer (`largemodel/utils/large_model_interface.py`)
 - Code Analysis
3. Practical Operations
 - 3.1 Configuring Offline Large Model
 - 3.1.1 Configuring LLM Platform (`yahboom.yaml`)
 - 3.1.2 Configuring Model Interface (`large_model_interface.yaml`)
 - 3.2 Startup and Testing

1. Concept Introduction

1.1 What is "Multimodal Visual Localization"?

Multimodal Visual Localization is a technology that combines multiple sensor inputs (such as cameras, depth sensors, IMUs, etc.) and algorithm processing techniques to achieve precise identification and tracking of the position and posture of devices or users in the environment. This technology does not rely on a single type of sensor data, but integrates information from different sensing modes, thereby improving the accuracy and robustness of localization.

1.2 Brief Description of Implementation Principles

1. **Cross-modal Representation Learning:** To enable LLMs to process visual information, a mechanism is needed to convert visual signals into a form that the model can understand. This may involve using Convolutional Neural Networks (CNNs) or other architectures suitable for image processing to extract features and map them to the same embedding space as text.
2. **Joint Training:** By designing appropriate loss functions, text and visual data can be trained simultaneously within the same framework, allowing the model to learn to establish connections between these two modes. For example, in a Q&A system, answers can be given based on either the provided text question or the relevant image content.
3. **Visually Guided Language Generation/Understanding:** Once an effective cross-modal representation is established, visual information can be used to enhance the capabilities of language models. For example, when given a photo, the model can not only describe what is happening in the image, but also answer specific questions about the scene, or even execute instructions based on visual cues (such as navigating to a certain location).

2. Code Analysis

Key Code

1. Tool Layer Entry (largemodel/utils/tools_manager.py)

The `visual_positioning` function in this file defines the execution flow of the tool, particularly how it constructs a Prompt containing the target object name and format requirements.

```
# From largemodel/utils/tools_manager.py
class ToolsManager:
    # ...
    def visual_positioning(self, args):
        """
        Locate object coordinates in image and save results to MD file.
        """
        self.node.get_logger().info(f"Executing visual_positioning() tool with
args: {args}")
        try:
            image_path = args.get("image_path")
            object_name = args.get("object_name")
            # ... (Path fallback mechanism and parameter checking)

            # Construct a prompt asking the large model to identify the
coordinates of the specified object.
            if self.node.language == 'zh':
                prompt = f"请仔细分析这张图片，用一个个框定位图像每一个{object_name}的位
置..."
            else:
                prompt = f"Please carefully analyze this image and find the
position of all {object_name}..."

            # ... (Build isolated message context)

            result = self.node.model_client.infer_with_image(image_path, prompt,
message=message_to_use)

            # ... (Process and parse returned coordinate text)

            return {
                "file_path": md_file_path,
                "coordinates_content": coordinates_content,
                "explanation_content": explanation_content
            }
        # ... (Error handling)
```

2. Model Interface Layer (largemodel/utils/large_model_interface.py)

The `infer_with_image` function in this file is the unified entry point for all image-related tasks.

```
# From largemodel/utils/large_model_interface.py

class model_interface:
    # ...
    def infer_with_image(self, image_path, text=None, message=None):
        """Unified image inference interface."""
        # ... (Prepare messages)
```

```

try:
    # Based on the value of self.llm_platform, determine which specific
    implementation to call
    if self.llm_platform == 'ollama':
        response_content = self.ollama_infer(self.messages,
image_path=image_path)
    elif self.llm_platform == 'tongyi':
        # ... Tongyi model logic
        pass
    # ... (Other platform logic)
    # ...
    return {'response': response_content, 'messages': self.messages.copy()}

```

Code Analysis

The core of the visual localization feature is **guiding the large model to output structured data through precise instructions**. It also follows the layered design of tool layer and model interface layer.

1. Tool Layer (`tools_manager.py`):

- The `visual_positioning` function is the business core of this feature. It receives two key parameters: `image_path` (image path) and `object_name` (name of the object to locate).
- The most critical operation of this function is **constructing a highly customized Prompt**. It doesn't simply ask the model to describe the image, but embeds `object_name` into a carefully designed template, explicitly requesting the model to "locate the position of every {object_name} in the image", and implicitly or explicitly requesting results in a specific format (such as coordinate arrays).
- After constructing the Prompt, it calls the model interface layer's `infer_with_image` method, passing both the image and this customized instruction.
- After receiving the returned text from the model interface layer, it also needs **post-processing**: using methods such as regular expressions to parse precise coordinate data from the model's natural language response.
- Finally, it returns the parsed structured coordinate data to the upper-level application.

2. Model Interface Layer (`large_model_interface.py`):

- The `infer_with_image` function continues to act as the "dispatch center". It receives the image and Prompt from `visual_positioning` and dispatches the task to the correct backend model implementation based on the current configuration (`self.llm_platform`).
- For visual localization tasks, the model interface layer's responsibility is essentially the same as for visual understanding tasks: properly package the image data and text instructions, send them to the selected model platform, and then return the text results unchanged to the tool layer. All platform-specific implementation details are encapsulated in this layer.

In summary, the general flow of visual localization is: `ToolsManager` receives the target object name and constructs a precise Prompt requesting coordinate output -> `ToolsManager` calls the model interface -> `model_interface` packages the image and Prompt together and sends them to the corresponding model platform based on configuration -> The model returns text containing coordinate information -> `model_interface` returns the text to `ToolsManager` -> `ToolsManager` parses the text, extracts structured coordinate data and returns it. This process demonstrates how to use Prompt Engineering techniques to enable general visual large models to complete more specific, more structured tasks.

3. Practical Operations

3.1 Configuring Offline Large Model

3.1.1 Configuring LLM Platform (`yahboom.yaml`)

This file determines which large model platform the `model_service` node loads as its primary language model.

1. Open the file in terminal:

```
vim ~/yahboom_ws/install/largemodel/share/largemodel/config/yahboom.yaml
```

2. Modify/confirm `llm_platform`:

```
model_service:                                #Model server node parameters
  ros__parameters:
    language: 'zh'                             #Large model interface language
    useolinetts: True                         #This setting is invalid in text
mode, can be ignored

# Large model configuration
llm_platform: 'ollama'                        # Critical: Ensure this is set to
'ollama'
```

3.1.2 Configuring Model Interface (`large_model_interface.yaml`)

This file defines which visual model is used when the platform is set to `ollama`.

1. Open the file in terminal

```
vim
~/yahboom_ws/install/largemodel/share/largemodel/config/large_model_interface.
yaml
```

2. Find the ollama-related configuration

```
#.....
## Offline Large Language Models
# Ollama configuration
ollama_host: "http://localhost:11434" # ollama server address
ollama_model: "llava"                # Critical: Change this to your downloaded
multimodal model, such as "llava"
#.....
```

Note: Please ensure the model specified in the configuration parameters (such as `llava`) can handle multimodal input.

3.2 Startup and Testing

1. Prepare image file:

Place a test image file at the following path:

```
/home/sunrise/yahboom_ws/src/largemodel/resources_file/visual_positioning
```

Then name the image `test_image.jpg`

2. Start the `largemodel` main program:

Open a terminal and run the following command:

```
ros2 launch largemodel largemodel_control.launch.py
```

3. Test:

- **Wake up:** Say to the microphone: "Hello, yahboom."
- **Conversation:** After the speaker responds, you can say: `analyze the position of the dinosaur in the image`
- **Observe logs:** In the terminal running the `launch` file, you should see:
 1. The ASR node recognizes your question and prints it out.
 2. The `model_service` node receives the text, invokes the LLM, and prints out the LLM's response.
- **Listen to the answer:** After a while, you should hear the answer from the speaker, and find an MD document recording the coordinate positions and localization object information under the path `/home/sunrise/yahboom_ws/src/largemodel/resources_file/visual_positioning.`